# Practical Work: Multi-Agent Communication

Implementation of FIPA-ACL standards with SPADE

Module: Distributed Artificial Intelligence (DAI) and MAS

Academic Year 2025-2026

**Objectives of the Practical Work**

The goal of this practical work is to put into practice the intentional communication concepts seen in class. You will manipulate the speech acts of the **FIPA-ACL** standard to make Python agents interact via the **SPADE** framework.

## 1 Introduction: From KQML to FIPA-ACL (Theory)

Historically, the **KQML** (Knowledge Query and Manipulation Language) laid the foundations for communication via speech acts. However, due to a lack of formal semantics, it was replaced by the **FIPA-ACL** standard.

In SPADE, we no longer use the LISP syntax of KQML, but we integrate FIPA-ACL intentions directly into the **metadata** of the messages.

**Reminder of the FIPA-ACL categories covered in this practical work:**

1. **Information Exchange:** `INFORM`, `QUERY-IF`
2. **Action Directives:** `REQUEST`, `AGREE`, `REFUSE`
3. **Negotiation (Auctions):** `CFP`, `PROPOSE`, `ACCEPT-PROPOSAL`
4. **Error Handling:** `FAILURE`, `NOT-UNDERSTOOD`

---

## 2 Exercise 1: Information Exchange (`QUERY-IF` and `INFORM`)

**Scenario:** A Monitor Agent (Manager) asks a Sensor Agent if the temperature exceeds a certain threshold.

### 2.1 Sensor Agent Code

The Sensor Agent uses a *Template* to only react to `QUERY-IF` type messages.

```python
from spade.agent import Agent
from spade.behaviour import CyclicBehaviour
from spade.template import Template

class SensorAgent(Agent):
    class BehaviorQuery(CyclicBehaviour):
        async def run(self):
            # The template ensures we only receive QUERY-IF
            msg = await self.receive(timeout=10)
            if msg:
                print(f"[Sensor] Request received: {msg.body}")

```

```
                # Preparation of the reply
                reply = msg.make_reply()
                reply.set_metadata("performative", "inform") # Information Category

                # Simulation of sensor reading
                temperature_actuelle = 25
                reply.body = f"The temperature is {temperature_actuelle}C"

                await self.send(reply)
                print("[Sensor] Reply (INFORM) sent.")

    async def setup(self):
        print("Sensor started.")
        # Strict FIPA-ACL filtering
        template = Template()
        template.set_metadata("performative", "query-if")
        self.add_behaviour(self.BehaviorQuery(), template)
```

### 2.2 Work to do

1. Create the script for the **Monitor Agent**. It must have a `OneShotBehaviour` that builds and sends a message with the `query-if` performative to the Sensor.
2. Run both agents and verify that the exchange takes place correctly.

---

## 3 Exercise 2: Action Directives (`REQUEST`, `AGREE`, `REFUSE`)

**Scenario:** The Monitor asks a Mobile Robot Agent to move. Unlike a classic object, the Robot agent is autonomous: it evaluates its internal state (its battery) before accepting or refusing.

### 3.1 Work to do

Modify the Robot Agent to implement the following decision logic in its `CyclicBehaviour`:

```
# Skeleton of the Robot's decision logic
async def run(self):
    msg = await self.receive(timeout=10)
    if msg: # If the received message is a REQUEST
        reply = msg.make_reply()

        # TODO: Check the agent's battery level
        if self.agent.battery_level > 20:
            # The agent accepts the order
            reply.set_metadata("performative", "agree")
            reply.body = "Movement authorized. Execution in progress."
            # TODO: Send the reply

            # TODO: Simulate the action (time.sleep), then send an INFORM(success)
            # or a FAILURE(obstacle) if an error occurs.
        else:
            # The agent refuses the order to protect itself
            reply.set_metadata("performative", "refuse")
            reply.body = "Error: Battery too low to move."
            # TODO: Send the reply
```

---

## 4 Exercise 3: Negotiation via Contract Net Protocol (`CFP`, `PROPOSE`)

**Scenario:** The Monitor Agent has a heavy computation task (e.g., training a CNN model). It makes a Call For Proposal (`CFP`) to several Worker Agents.

### 4.1 Work to do

1. **The Call for Proposals:** Code the Monitor Agent so that it sends a FIPA-ACL message of type `cfp` to two Worker Agents. The body of the message (`body`) will contain the task name: "CNN Training".

2. **The Proposals:** Each Worker Agent receives the `cfp`.

   - Worker 1 replies with a `propose` (Cost: 10 time units).
   - Worker 2 replies with a `propose` (Cost: 5 time units).

3. **The Allocation:** The Monitor receives the proposals, compares them, and sends an `accept-proposal` to Worker 2, and a `reject-proposal` to Worker 1.

**Filtering:** The Monitor Agent will have to use a `Template` to listen specifically for messages whose performative metadata is `"propose"`.